

Chapter 11

CS2 course package - Tennessee Tech University (TNTECH)

April R. Crockett

Tennessee Technological University
acrockett@tntech.edu

Abstract

This chapter presents a transformation of the second course in the introductory programming sequence (CS2) at Tennessee Technological University in the Department of Computer Science named "Data Structures and Algorithms." This chapter also provides access to course materials so that other institutions may be able to adopt and use the materials. Parallel and Distributed Computing (PDC) concepts were introduced into all sections of CS1 starting in Spring 2025 through lecture discussions and (plugged) programming assignments. The primary learning outcomes are for students to (1) recognize terminology associated with parallel and distributed computing topics, (2) identify algorithmic solutions that are representative of parallelism, and (3) develop programming solutions that are representative of data parallelism.

Descriptor Elements

Programming Language Employed: C++

Relevant core courses: CS2

Pre-requisites: CS1

Textbook: Open Access Resources

Relevant PDC topics and Blooms level Below is a list of Parallel Computing topics that we focused on when designing and creating materials for CS2.

- Data & Task Parallelism (Blooms levels: Understand, Analyze & Apply)

Learning outcomes: Table 11.1 shows the learning outcomes added to Tennessee Tech's CS2 course with the incorporation of Parallel Computing topics.

Table 11.1: TNTECH CS2 Learning Outcomes & Blooms Levels

Learning Outcomes	Blooms Level
Recognize parallel computing terms such as data parallelism, task parallelism, task decomposition, speedup, scalability, and pipelining.	Level 2: Understand
Demonstrate the ability to identify algorithmic solutions that are representative of data parallelism.	Level 4: Analyze
Implement algorithmic solutions that are representative of data parallelism.	Level 3: Apply

Context for use: Tennessee Technological University (TNTECH) is a mid-sized public research university in Cookeville, Tennessee, United States. With an enrollment of about 10,000 students, TNTECH offers more than 40 undergraduate and 20 graduate degree

programs across eight academic colleges and schools. The Department of Computer Science at TNTECH offers Bachelors, Masters, and Ph.D. degrees and consists of approximately 700 total students. At present, we have an average of about 120 students each semester enrolled in our “Data Structures & Algorithms” (CS2) course. The course is a 4 credit-hour lecture-lab, with 3 hours devoted to lecture and 1 hour devoted to lab. The course is usually split into two sections of lecture and four to six sections of lab, where each lab section consists of approximately 25 students each. C++ is the programming language used for instruction, and this course is the second required computer science class in the curriculum of the Computer Science and AI undergraduate degree programs. The CS2 course is taken by students in several additional majors as well, as shown in Table 11.2. Eighty percent of the students in CS1 identify as male and approximately 15% identify as female as shown in Table 11.3 and student race is shown in Table 11.4. Table 11.5 shows the classification of students taking the CS1 course.

Table 11.2: TNTECH CS2 Majors

Major	Spr 2025 N=52	Fall 2025 N=116	Spr 2026 N=151	Total N=319
Business Info. Tech.	0	1	0	1 (0.3%)
Computer Engineering	2	16	19	37 (11.6%)
Computer Science	40	52	70	162 (50.8%)
Electrical Engineering	8	41	53	102 (32.0%)
Environmental Engineering	1	0	0	1 (0.3%)
Finance	0	0	1	1 (0.3%)
Mathematics	2	2	5	9 (2.8%)
Mechanical Engineering	0	1	2	3 (0.9%)
Music	0	0	1	1 (0.3%)
Nuclear Engineering	0	1	1	2 (0.6%)
Physics	1	3	8	12 (3.8%)
Spanish	1	0	0	1 (0.3%)

Table 11.3: TNTECH CS2 Student Gender Identity

Gender	Spr 2025 N=52	Fall 2025 N=116	Spr 2026 N=151	Total N=319
Male	35	91	121	247 (77.4%)
Female	13	14	19	46 (14.4%)
Other	1	2	1	4 (1.3%)
No Response	3	9	10	22 (6.9%)

Table 11.4: TNTECH CS2 Race/Ethnicity of Students

Race/Ethnicity	Spr 2025 N=52	Fall 2025 N=116	Spr 2026 N=151	Total N=959
White	31	78	110	219 (68.7%)
Asian/ Pacific Islander	10	6	9	25 (7.8%)
Black/ African American	6	3	12	21 (6.6%)
Prefer Not to Respond	3	11	4	18 (5.6%)
Hispanic	1	7	9	17 (5.3%)
2 or More Ethnicities	0	9	6	15 (4.7%)
American Indian/ Alaska Native	0	1	1	2 (0.6%)
Middle Eastern/ North African	0	1	0	1 (0.3%)

Table 11.5: TNTECH Classification of CS2 Students

Classification	Spr 2025 N=51	Fall 2025 N=115	Spr 2026 N=150	Total N=316
First year (freshman)	24	3	55	82 (25.9%)
Sophomore	17	81	62	160 (50.6%)
Junior	8	27	29	64 (20.3%)
Senior	2	3	4	9 (2.8%)
Graduate student	0	1	0	1 (0.3%)

11.1 Introduction

Modern computing increasingly relies on parallelism, distributed systems, and data-intensive workflows. National curriculum recommendations such as the ACM/IEEE Computer Science Curricula and the IEEE TCPP Parallel and Distributed Computing (PDC) guidelines, have highlighted the growing relevance of these topics within undergraduate computing education [? ?]. At Tennessee Technological University, we have infused parallel and distributed computing (PDC) concepts into our undergraduate computing curricula starting with our CS1 course and continuing on in our CS2 course.

Section 11.2 below describes in detail what was changed in our CS2 course and what specifically was added. We modified a few of our lectures and modified some assignments to incorporate Parallel Computing topics. Section ?? highlights the changes and section 11.2.4 discusses challenges faced in updating the curriculum. The details on all new activities is in section 11.3.

11.2 How CS2 was changed and PDC topics integrated

To add PDC topics in CS2 at TNTECH, we modified an existing assignment so that traditional CS2 objectives were retained while parallel computing concepts were embedded

into learning sort algorithms. Specifically, a programming assignment formerly centered on students writing and comparing time efficiencies of sort algorithms was revised so that students used OpenMP to additionally write and compare parallel sort algorithms.

11.2.1 PDC Topics Integrated

In our CS2 course, parallel programming and OpenMP was the lecture topic during week 4 of lecture and practiced via a programming assignment during weeks 5 through 7.

11.2.2 Integrated Syllabi (Pre and Post)

There is a syllabus and master schedule for each semester in the GitHub repository (https://github.com/cscaprilcrockett/CDER_TNTECH_CS2/tree/main/Syllabi). CS2 is named CSC 1310 at TNTECH. The schedule and syllabus for Spring 2025 is before PDC topics were introduced and all other semesters are after.

11.2.3 Highlights

11.2.4 Challenges

Assignment that Didn't Work

In Fall 2026 we tried incorporating accessing data from a remote service via a lab assignment in the course. The assignment ended up being too open-ended. However, with some modifications, I plan to add this assignment back to the course. The problem was that the assignment was too open-ended in allowing students to find an JSON data repository to use but many students ran in the problem of finding repositories that required a key or repositories with data issues.

Preparing the Teaching Team for New Territory

Another challenge was that the new material was not only new to the students in the course, but also to the team of teaching assistants. This meant that I would first need to find time to teach the concepts and assignments to the teaching assistants before teaching the students. Finding the time to do this was definitely challenging. There were also some teaching assistants that didn't understand why these concepts were being taught in CS2 and thought it was too much to ask CS2 students to learn about and practice these concepts. I originally didn't expect this confusion and didn't explain the reasoning. However, in the 2nd semester of implementing the PDC concepts I knew that explaining the reasoning must be part of the TA training.

11.3 New Plugged-in Activity

11.3.1 Sort Algorithm Time Comparison Programming Assignment

Problem Description, Prerequisites, and Learning Outcomes

The **prerequisites** for this programming assignment include understanding algorithm analysis and sort algorithms. At Tennessee Tech (TNTECH), we cover these topics in week 4 (out of 16) and then students are assigned this 3-week long programming assignment during week 5 of the semester.

Figure 11.1: TNTECH CS2 Program 2

PROGRAM 2

CSC 1310, SPRING 2026

SORTING ALGORITHM COMPARISON

OBJECTIVE OF THIS ASSIGNMENT



The primary goal & challenge of this assignment is not just learning to write code but in learning about sort algorithms, comparing algorithm efficiencies, and learning to apply parallel programming using OpenMP.

Modern software is built to run fast and efficiently on multi-core systems, and that means parallel programming is **no longer optional—it's essential**. Learning it early **gives you a serious edge**, making you more competitive, job-ready, and prepared to solve real-world problems at scale.

Image created with ChatGPT 4.0 https://chatgpt.com/s/m_6868273bc4b48191978c1a469a4cb100

TWO DUE DATES!

- Part 1: The working program code is due on **3/3/2026**
- Part 2: The testing documentation document is due on **3/5/2026**

Students filled out a program report after every programming assignment. Students rate their level of agreement with the three statements shown in Table 11.6. Overall, students responded positively to the assignment, with the strongest result being that students felt they learned from the assignment. A large majority of students agreed at least somewhat that they learned from it, with 87.2% selecting Somewhat Agree, Agree, or Strongly Agree, and only 1.4% expressing disagreement. Students were also generally confident in their performance, with 81.8% agreeing that they thought they made a good grade.

Several open-ended questions were also asked. One of the questions asked was "What is something you learned from the assignment?" Students' responses suggest that the assignment primarily helped them understand OpenMP, parallel programming, sorting algorithms, and performance tradeoffs. The strongest theme was that many students learned how to apply OpenMP syntax and parallelize code, especially through `#pragma`, parallel sections,

Figure 11.2: TNTECH CS2 Program 2 Part 1

GIVEN CODE

- `program2.cpp`
- `program2.h`
- `numsSmall.txt` - use this text file along with `wordsSmall.txt` to test your sorting functions to make sure they actually sort correctly
- `wordsSmall.txt` - use this text file along with `numsSmall.txt` to test your sorting functions to make sure they actually sort correctly
- `numsLarge.txt`
- `wordsLarge.txt`

SPECIFICATIONS

GIVEN CODE

In `program2.cpp`, you are given all the `#includes`, part of the `main` function, the `readFromFile()` function, the `printVector()` function, and the `scramble()` function.

MAIN FUNCTION

You are given some of this function and then there are detailed comments describing what you should add in this function.

PROTOTYPE THE FUNCTIONS AS YOU IMPLEMENT THEM

In the `program2.h` file, write function prototypes for the functions listed below. All these functions should be template functions because they need to be able to be used with both integer vectors and string vectors.

- `bubbleSort`: The input will be a vector
- `parallelBubbleSort`: The input will be a vector
- `mergeSort`: The input will be a vector, the low index and the high index.
- `parallelMergeSort`: The input will be a vector, the left index, the right index, and a depth integer variable for parallelization
- `merge`: The input will be the vector, the left index, the right index, and the middle index. This function will be called by both `mergeSort` and `parallelMergeSort`.

BUBBLE SORT

For the `bubbleSort()` function, you will implement the Bubble Sort algorithm as shown in class and in sample code.

PARALLEL "BUBBLE SORT"

Technically, once you alter the Bubble Sort algorithm to be able to run in parallel, it is really no longer bubble sort but instead is the "Odd Even Transposition Sort" or "Brick Sort". In this algorithm we alternately swap the even items with their next item for one pass, and then the odd items with their next item for the next pass. It is similar to another sort algorithm called the `shaker sort`. This algorithm lends itself well to parallelization because all of the compare-and-swap operations in each pass could theoretically be performed at once - on a vector using OpenMP to accelerate the process by taking advantage of multiple CPU threads.

- Before writing code, check out this visualization first: <https://sortvisualizer.com/oddeven-sort/> (turn sound up) 
- Notice that elements are still "bubbling up" - but in a different way.
- You can read more about the odd-even sort here: https://en.wikipedia.org/wiki/Odd%E2%80%93even_sort
- And you can see another visualization (not as fun one, though) and see sample (Java) code here: <https://algorithms.wtf/visualizations/sorting/oddeven-sort.php>

Notice in the Java code example from the last website I provided above that there are TWO for loops - one for odd indexes and the other for even indexes. Both of these for loops are nested in a while loop.

To set up this algorithm to run in parallel, you will have to have define a parallel region INSIDE the while loop

```
#pragma omp parallel
```

(and enclose the parallel region in curly braces). The parallel region should contain the two for loops.

However, just setting up the parallel region isn't enough to make the for loops be able to run through the vector elements in parallel.

You will need to use `#pragma omp for reduction(;;:swapped)` above each for loop. The "swapped" part of this `#pragma` is just the Boolean variable that I used in order to exit the loop if no swaps were made in the last iteration of the while loop. In the Java code in the [algorithms.wtf](https://algorithms.wtf/visualizations/sorting/oddeven-sort.php) website, they used the variable "sorted". This `#pragma` directive uses OpenMP's logical OR (||) `reduction` which causes each thread to get a private copy of the swapped variable. Each thread will work independently and update its own private copy. At the end of the parallel region (end of the for loop), OpenMP combines all the private copies using the operator specified (in this case, || = logical OR) and this is what we want because if any thread sets its swapped to true, the final shared swapped becomes true and means that we aren't done sorting the vector so keep working. The logical OR reduction prevents every thread from modifying the shared variable directly, which would cause the following:

- Race conditions (multiple threads trying to write to swapped at the same time)
- False sharing and cache coherence issues
- Overhead from synchronizing writes to shared memory (this slows things down and creates unpredictable behavior)

MERGE SORT

For the `mergeSort()` function, you will implement the Merge Sort algorithm as shown in class and example code, which means you will also need a `merge()` function. The `merge()` function is used by both the `mergeSort()` function and the `parallelMergeSort()` function.

PARALLEL MERGE SORT

When you call the `parallelMergeSort()` function from the main function, you will be sending a fourth argument that represents the depth of recursion, which begins at zero. The depth is required in the parallel version of this algorithm because depth controls how deep the parallel recursion goes to prevent creating too many threads, which would end up slowing down runtime instead of speeding up!

After calculating the middle index of the current range to split the vector into two halves, you will then need an `if` statement to check if the depth is below 4. This is because if we are not too deep in the recursion tree, we allow parallelism. This keeps thread creation under control.

If the depth is below 4, then you will have an OpenMP directive that will tell OpenMP to run independent code `sections` in parallel using multiple threads. Refer to the OpenMP slides on "Task Parallelism with Parallel Sections". Each recursive call to `parallelMergeSort()` should be preceded by an OpenMP directive that will define an OpenMP section. This means that the left half of the vector (from left to mid) will be in a new thread (section) and the right half of the vector (from mid+1 to right) will also be in a new thread (section).

If the depth is at 4 or above, then just do the recursive calls to `parallelMergeSort()` without parallelizing them.

Table 11.6: TNTECH CS2 Program 2 Feedback from Students

Statement	Agree (Strongly Agree, Agree, Somewhat Agree)	Neutral	Disagree (Somewhat Disagree, Disagree, Strongly Disagree)
I enjoyed this assignment	110 (74.3%)	27	11 (7.4%)
I learned from this assignment	129 (87.2%)	17	2 (1.4%)
I think I made a good grade on this assignment	121 (81.8%)	20	7 (4.7%)

task parallelism, thread usage, and compiler flags such as `-fopenmp`. A second major theme was that students deepened their understanding of sorting algorithms, particularly bubble sort, merge sort, quicksort, and how different algorithms behave with large datasets. Several students said that implementing the algorithms themselves helped them better understand how the algorithms work, how loops and variables function within them, and how sorting methods differ in efficiency. Another important finding is that many students learned that parallelization does not automatically improve performance. Several responses specifically noted that parallel code can be slower than serial code, especially for small datasets or when thread creation, synchronization, or coordination overhead outweighs the benefit. Students also observed that more threads do not always mean faster runtime, and that there may be an optimal number of threads depending on the algorithm and dataset size. Students also

Figure 11.3: TNTECH CS2 Program 2 Part 2

PART 2: DOCUMENTATION

In a Word document named **program2_documentation.docx**, you must document going through the following steps to test your code **after you know it is working properly**. Read incremental programming & testing above!

1. First, make sure to **disable anti-virus** on your computer and **stop/close all unnecessary programs** from running (close out Teams, Outlook, Discord, etc.). You may keep Microsoft Word or Google Docs open to document the steps below.
2. **Run Sample Output 1** (shown in Sample Output section in this document).
3. Take a **screenshot** of your sample output 1 results and paste into the document.
4. **Analyze your results** and write answers to the questions below in the document.
 - a. Why do you think you got the results that you got?
 - b. What algorithm was the fastest out of the algorithms you implemented?
 - c. What algorithm was the slowest?
 - d. Was the parallel bubble sort algorithm faster than the serial bubble sort algorithm?
 - e. How does your results compare to the sample output provided?
5. **Run Sample Output 2** (shown in next section), including the input validation.
6. Take a **screenshot** of your sample output 2 results and paste into the document.
7. **Analyze your results**.
 - a. Why do you think you got the results that you got?
 - b. How does it compare to the sample output provided?
8. **Run Sample Output 3** (shown in next section).
9. Take a **screenshot** of your sample output 3 results and paste into the document.
10. **Analyze your results**.
 - a. Why do you think you got the results that you got?
 - b. How does it compare to the sample output provided?
11. **Try several different numbers of threads** and figure out what number of threads gave you the quickest runtime for the parallel algorithms. Provide a **screenshot** of the quickest runtime of parallel bubble sort.
12. You may have noticed that when you increase the number of threads that parallel bubble sort slows down significantly, but parallel merge sort does not. **Why is that?** (Note: you may use ChatGPT or similar GenAI to help you with this answer if needed).

reported learning how to measure and compare runtime, including use of the chrono library, best-case and worst-case runtime differences, and how performance can vary across devices. A few students connected the assignment to broader ideas about algorithm efficiency, data type effects, and the strengths and weaknesses of different sorting approaches. There were also several responses indicating that the assignment was challenging. Some students described OpenMP or parallel programming as confusing, strange, difficult, or frustrating, while still acknowledging that they were improving. Overall, the responses indicate that the assignment was successful in helping many students connect sorting algorithm implementation with parallel programming concepts and empirical performance testing. The most meaningful learning outcome appears to be that students moved beyond seeing parallelism as simply “faster” and began recognizing that performance depends on the algorithm, dataset size, number of threads, overhead, and hardware.

Instructional Material (Lectures, Assignments)

- Lecture Materials on Algorithm Analysis, Sort Algorithms, and Parallel Programming:
https://github.com/cscaprilcrockett/CDER_TNTECH_CS2/tree/mai

n/Lecture%20Materials

- Assignment Document and Files Given to Students: https://github.com/cscaprilcrockett/CDER_TNTECH_CS2/tree/main/Plugged%20Activities/Sorting%20Algorithms%20with%20OpenMP%20Programming%20Assignment

Data and Analysis

11.4 Course-wide Pre and Post Course Surveys

To determine if our new and modified modules and assignments were effective at teaching basic PDC concepts while not reducing understanding of core, fundamental CS2 topics, we administered pre- and post-course surveys. To download the actual surveys performed in CS2 at TNTECH, go to https://github.com/cscaprilcrockett/CDER_TNTECH_CS2/tree/main/Data/Surveys.

Table ?? shows how many students were enrolled in the CS2 course from Spring 2025 to Spring 2026. The table also shows how many students filled out the Pre-Course survey and Post-Course survey each semester who also consented to share their data in the study. These students' results are those reported in Section 11.4.1.

Table 11.7: TNTECH CS2 Sample Size for Pre-Course and Post-Course Surveys

Population	Spr 2025	Fall 2025	Spr 2026	Total
No. Students Enrolled in CS1 Course	60	132	168	360
No. Consented Students Pre-Course Survey	52	116	151	319 (88.6%)
No. Consented Students Post-Course Survey	51	115	150	316 (87.8%)

In both the pre-course and post-course surveys, students were asked several 7-point Likert scale questions where the possible responses were:

- No Experience At All
- Very Little Experience
- Slightly Experienced
- Between Slightly and Moderately Experienced
- Moderately Experienced
- Very Experienced
- Extremely Experienced

The first set of questions asked students to rate their experience with using the following in their programs:

- Classes & Objects
- Searching algorithms
- Sorting algorithms
- Recursion
- Template Classes/ Generics
- Lists/ Linked Lists
- Stacks
- Queues
- Priority Queues
- Hashing/ Hash Tables
- Trees
- Binary Search Trees
- Balanced Trees/ AVL Trees
- Heaps
- Treap
- Graphs
- B-Trees
- Sets
- Associative Array, Map, or Dictionary
- Writing test cases
- Big-O notation
- Choosing the correct data structure
- Writing classes
- Problem decomposition: Breaking the desired functionality into separate parts (e.g. functions) that can be implemented separately

The second set of questions asked students to rate their experience with the following:

- **Parallel processing:** Computing where the operations within a program are being executed simultaneously (i.e., running on different cores of the processor at the same time)
- **Distributed computing:** Computing with a program that runs on multiple computers or that runs using data from a different computer (i.e., an online service)
- **Event handling:** Having a function that responds to an independent action (i.e., a mouse click)

11.4.1 Results of Pre-Course and Post-Course Surveys

Growth in Core Programming Concepts

As shown in Table 11.8, across the Spring 2025 through Spring 2026 intervention period, students reported large pre-to-post gains in core data structures and algorithm concepts. We had 285 students complete both the pre-course and post-course surveys. In this Table, Pre Mean is the mean pre-course response score for the students included in that item's analysis. Post Mean is the mean post-course response score for the same students included in the pre-course mean. Diff is the mean paired change from pre-course to post-course, calculated as Post Mean - Pre Mean. Positive values indicate an increase from pre to post. Wilcoxon is the Wilcoxon signed-rank test statistic for the paired pre/post responses. This nonparametric test evaluates whether the median paired change differs from zero. The p-value from the Wilcoxon signed-rank test is also displayed in the table. Smaller values indicate stronger evidence that the pre/post change was not due to random variation. The "sig?" column indicates whether the Wilcoxon test result was statistically significant (yes or no) using alpha level .05. A value of yes means p is less than 0.05 and no means p is greater than or equal to .05.

Growth in PDC Concepts

As shown in Table 11.9, students reported significant gains in parallel processing (largest gain), distributed computing, and event handling (smallest gain). In the table, Pre Mean is the mean pre-course response score for the students included in that item's analysis. Post Mean is the mean post-course response score for the same students included in the pre-course mean. Diff is the mean paired change from pre-course to post-course, calculated as Post Mean - Pre Mean. Positive values indicate an increase from pre to post. Wilcoxon is the Wilcoxon signed-rank test statistic for the paired pre/post responses. This nonparametric test evaluates whether the median paired change differs from zero. The p-value from the Wilcoxon signed-rank test is also displayed in the table. Smaller values indicate stronger evidence that the pre/post change was not due to random variation. The "sig?" column indicates whether the Wilcoxon test result was statistically significant (yes or no) using alpha level .05. A value of yes means p is less than 0.05 and no means p is greater than or equal to .05.

Table 11.8: TNTECH CS2 Student Comfort With Core Topics

Topic	Pre Mean	Post Mean	Diff	Wilcoxon	p-value	sig?
Classes & Objects	3.98	5.63	1.65	287.00	1.65E-38	Yes
Searching algorithms	2.39	5.11	2.72	346.00	5.12E-44	Yes
Sorting algorithms	2.39	5.11	2.72	427.00	8.64E-44	Yes
Recursion	2.24	4.89	2.65	617.00	8.98E-42	Yes
Template Classes/ Generics	2.14	5.11	2.97	315.00	4.37E-45	Yes
Lists/ Linked Lists	2.44	5.46	3.02	427.00	4.95E-44	Yes
Stacks	1.76	5.31	3.55	99.00	1.83E-47	Yes
Queues	1.64	5.26	3.62	118.50	2.31E-47	Yes
Priority Queues	1.53	4.44	2.91	268.00	3.99E-45	Yes
Hashing/ Hash Tables	1.60	4.86	3.26	174.00	6.04E-47	Yes
Trees	1.64	5.08	3.45	114.00	5.25E-47	Yes
Binary Search Trees	1.59	5.09	3.50	168.00	7.74E-47	Yes
Balanced Trees/ AVL Trees	1.45	4.63	3.18	54.00	2.15E-46	Yes
Heaps	1.56	4.28	2.72	42.00	4.94E-45	Yes
Treap	1.37	3.53	2.17	345.00	1.84E-39	Yes
Graphs	1.76	4.85	3.09	104.50	7.12E-46	Yes
B-Trees	1.38	3.75	2.37	51.00	1.81E-41	Yes
Sets	1.68	3.65	1.97	977.00	9.37E-35	Yes
Associative Array, Map, or Dictionary	1.84	3.93	2.08	905.00	5.78E-37	Yes
Writing test cases	2.07	4.61	2.54	618.00	9.03E-42	Yes
Big-O notation	1.69	4.87	3.18	282.50	2.67E-46	Yes
Choosing the correct data structure	2.38	5.27	2.89	323.00	3.50E-44	Yes
Writing Classes	3.07	5.74	2.68	75.00	8.61E-45	Yes
Problem decomposition	3.64	5.46	1.82	1280.00	5.58E-35	Yes

Conclusion of Results

The overall survey results suggest that students made significant pre-to-post gains across both traditional CS2 topics and Parallel Computing concepts.

11.5 Conclusion

While most PDC efforts have focused on programming strategies for upper-level and elective courses, we believe there is value and opportunity in introducing these concepts much earlier in the computing curriculum. Our studies have demonstrated that our approach has statistically significant and substantial gains in both learning and appreciation for PDC

Table 11.9: TNTECH CS2 Student Comfort With PDC Topics

Topic	Pre Mean	Post Mean	Diff	Wilcoxon	p-value	sig?
Parallel Processing: Computing where the operations within a program are being executed simultaneously	2.68	4.40	1.73	1102.00	1.94E-37	Yes
Distributed Computing: Computing with a program that runs on multiple computers or that uses data from a different computer	2.28	3.98	1.70	1492.50	8.26E-34	Yes
Event Handling: Having a function that responds to an independent action	2.74	4.21	1.47	2905.00	1.26E-25	Yes

topics. We hope that other educators can adopt or adapt our modules and activities and replicate our results and success.

Bibliography

Appendix

|You must include these four specific subsections (in addition to anything else):|

Github Link to Instructional Material

https://github.com/cscaprilcrockett/CDER_TNTECH_CS2

Organization of Material

The Github materials for CS2 are organized as follows:

- **Data/Surveys** - contains all the surveys given to CS2 students during Spring 2025, Fall 2025, and Spring 2026 semesters.
- **Lecture Materials** - contains worked examples and PowerPoint presentation slides for each module discussed in this chapter.
- **Plugged Activities** - contains programming and lab assignments (without solutions) to the assignments discussed in this chapter.
- **Syllabi** - contains the *Syllabus* and *Master Schedule* from Spring 2025, Fall 2025, and Spring 2026. The "Master Schedule" contains details on what was covered each day in lecture and the lab for the week. It also has an additional tab that details the contents and goals for each module in the course.